Open Access | https://doi.org/10.25080/eyxa7843



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Detecting Anomalies in Scientific Data Using SciPy's Statistical and Signal Tools

Aayush Gauba[1]

[1]Southern Illinois University Edwardsville

Abstract

Scientific datasets often contain subtle anomalies caused by sensor faults, experimental drift, measurement noise, rare events, or abnormal signal behavior. In many scientific workflows, anomaly detection must be not only accurate but also interpretable, because researchers need to understand whether unusual observations reflect meaningful phenomena, instrument error, or changes in experimental conditions. This paper presents a SciPy-based anomaly detection workflow using statistical and signal-processing tools. Rather than evaluating the workflow only on a single benchmark dataset, we construct a controlled multi-domain experiment using six scientific signal domains: electroencephalography (EEG)-like signals, environmental sensor signals, spectroscopy-like signals, vibration signals, astronomical photometry-like signals, and fluid-sensor signals. Five controlled anomaly mechanisms are injected into these signals: spike, drift, dropout, oscillation disturbance, and distribution shift. SciPy statistical, signal-processing, residual, spectral, local-window, and complexity features are extracted from every signal. The experiment evaluates binary anomaly detection, anomaly-mechanism classification, relative feature-group signatures, and rule-based diagnosis. In the controlled experiment, SciPy-derived features achieved perfect binary anomaly detection, but anomaly-mechanism classification was substantially harder, with the best SciPy-feature model achieving macro-F1 = 0.4578 across five anomaly types. A hand-written rule-based diagnosis system using relative feature-group signatures achieved only 0.1993 accuracy, approximately chance. This shows that detection performance does not imply diagnostic interpretability. To test whether the workflow transfers beyond controlled injection, we also evaluate it on the University of California, Riverside (UCR) Time Series Anomaly Archive [1]. Because the controlled benchmark provides known anomaly labels while the UCR archive provides only normal training regions, the two experiments evaluate complementary supervised and unsupervised settings. Across 56 evaluated real anomaly files, the best SciPy method achieved mean point-adjusted F1 = 0.2884, receiver operating characteristic area under the curve (ROC-AUC) = 0.7758, precision-recall area under the curve (PR-AUC) = 0.1801, and event detection rate = 0.4107. These results show that SciPy features provide a useful and transparent anomaly-detection baseline, but also reveal the limits of global feature summaries: real anomaly detection remains difficult, and mechanism diagnosis requires more than simple feature-group explanations.

Keywords anomaly detection, scipy, time series, statistical diagnostics, signal processing

Published Jul 13, 2026

Correspondence to
Aayush Gauba
gauba.aayush@gmail.com

Open Access

Copyright © 2026 Gauba. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Scientific data is rarely perfectly clean. Measurements collected from sensors, laboratory instruments, simulations, biological systems, telescopes, and physical experiments may contain unexpected behavior caused by noise, sensor faults, experimental drift, environmental interference, or rare events. Detecting these anomalies is important because abnormal observations can distort downstream analysis, reduce model reliability, and lead to incorrect scientific conclusions.

However, anomaly detection in scientific data is not only a prediction problem. It is also a diagnostic problem. A researcher often needs to know why a signal is unusual. For example, an abnormal measurement may be caused by a transient spike, a gradual drift, a dropout or flatline, a spectral disturbance, or a distribution shift. A model that only reports "anomaly" may be useful for flagging data, but it may not provide enough information to support scientific interpretation or corrective action.

This paper studies anomaly detection using SciPy's statistical and signal-processing tools. SciPy is a foundational scientific computing library for Python that provides algorithms for statistics, signal processing, optimization, integration, interpolation, linear algebra, and other scientific computing tasks [2]. The `scipy.stats` module provides probability distributions, summary statistics, and statistical tests. The `scipy.signal` module provides tools for filtering, peak detection, spectral analysis, and time-series signal processing. These tools make SciPy a practical foundation for anomaly detection workflows that need to be lightweight, reproducible, and interpretable.

We test whether SciPy-derived features detect controlled anomalies and support diagnosis. Six domain-inspired signal generators, five anomaly mechanisms, and multiple severity and noise levels provide known ground truth. We then test transfer on the UCR archive. Our central claim is that detection performance does not imply diagnostic interpretability: global energy features respond to several mechanisms, so diagnosis requires localized or mechanism-specific information.

The contributions are:

- a controlled multi-domain anomaly benchmark using six scientific signal domains and five anomaly mechanisms;
- a SciPy-based feature extraction workflow using statistical, signal-processing, spectral, residual, local-window, and complexity features;
- a comparison of SciPy-feature models, relative-signature models, and raw-signal baselines;
- comparisons of feature, signature, and raw-signal models for detection and diagnosis;
- UCR external validation;
- evidence that dominant-feature-group explanations are insufficient for diagnosis.

2. Related Work

Time-series anomaly detection has a long history in statistical process monitoring. Classical approaches include threshold-based monitoring, Shewhart control charts, cumulative sum methods (CUSUM), and exponentially weighted moving average methods. These approaches are attractive because they are simple, interpretable, and computationally lightweight, but they often require assumptions about stationarity, distributional behavior, or known control limits.

Unsupervised machine-learning methods provide a more flexible alternative when labeled anomalies are rare. Isolation Forest is a widely used representative method that isolates anomalies through randomized partitioning rather than explicitly modeling normal density [3]. One-class methods such as One-Class SVM similarly learn a boundary around normal data and are commonly used when only normal training samples are available [4]. These methods are relevant to the UCR validation setting in this paper, where training regions are treated as normal and anomaly labels are used only for evaluation.

Deep learning methods include LSTM encoder-decoder reconstruction models [5] and transformer-based models such as TranAD [6]. They can be powerful, but often require more data, tuning, and computation than classical feature-based methods.

Benchmark design is also a central issue in anomaly detection. Wu and Keogh argue that many time-series anomaly detection benchmarks are flawed and can create an illusion of progress, motivating more careful evaluation protocols and realistic anomaly archives [1]. This concern is directly relevant to the present work: perfect detection on a controlled synthetic benchmark should not be interpreted as deployment-level success. For this reason, the controlled benchmark in this paper is used for mechanism testing, while the UCR archive is used as an external validation setting.

This paper differs from prior work by focusing on the distinction between anomaly detection and anomaly diagnosis. The goal is not to establish a new state-of-the-art detector. Instead, we test whether SciPy-derived statistical and signal-processing features provide a transparent representation that can detect anomalies, preserve partial mechanism-level information, and expose the limitations of simple feature-group interpretability.

3. Methods

Table 1 summarizes the methodological difference between the controlled and UCR evaluations.

Table 1. *Learning settings used in the two evaluation stages.*

Experiment	Training Signal	Evaluation Goal
Controlled benchmark	Labeled anomalies	Detection and diagnosis
UCR validation	Normal windows only	Real-data detection

3.1. Scientific Signal Domains

We generate clean signals from six scientific signal domains:

- **EEG-like signals**: mixtures of low-frequency rhythms, burst-like oscillations, and colored noise;
- **Environmental sensor signals**: slow periodic cycles, low-frequency trends, and correlated noise;
- **Spectroscopy-like signals**: smooth baselines with Gaussian absorption or emission peaks;
- **Vibration signals**: oscillatory signals with harmonics and amplitude modulation;
- **Astronomical photometry-like signals**: light-curve-like signals with periodic dips and correlated noise;
- **Fluid-sensor signals**: quasi-periodic vortex-shedding-like signals with modulation and noise.

These are synthetic domain-inspired signals, not measurements from real instruments. They are used because controlled generation allows the true anomaly mechanism, severity, and noise level to be known. These domains were chosen because they differ in baseline structure, noise statistics, and signal dynamics. This allows the experiment to test whether anomaly behavior generalizes across heterogeneous scientific signals. Figure 1 shows one independently sampled clean trace and one drift-anomaly trace from each domain.

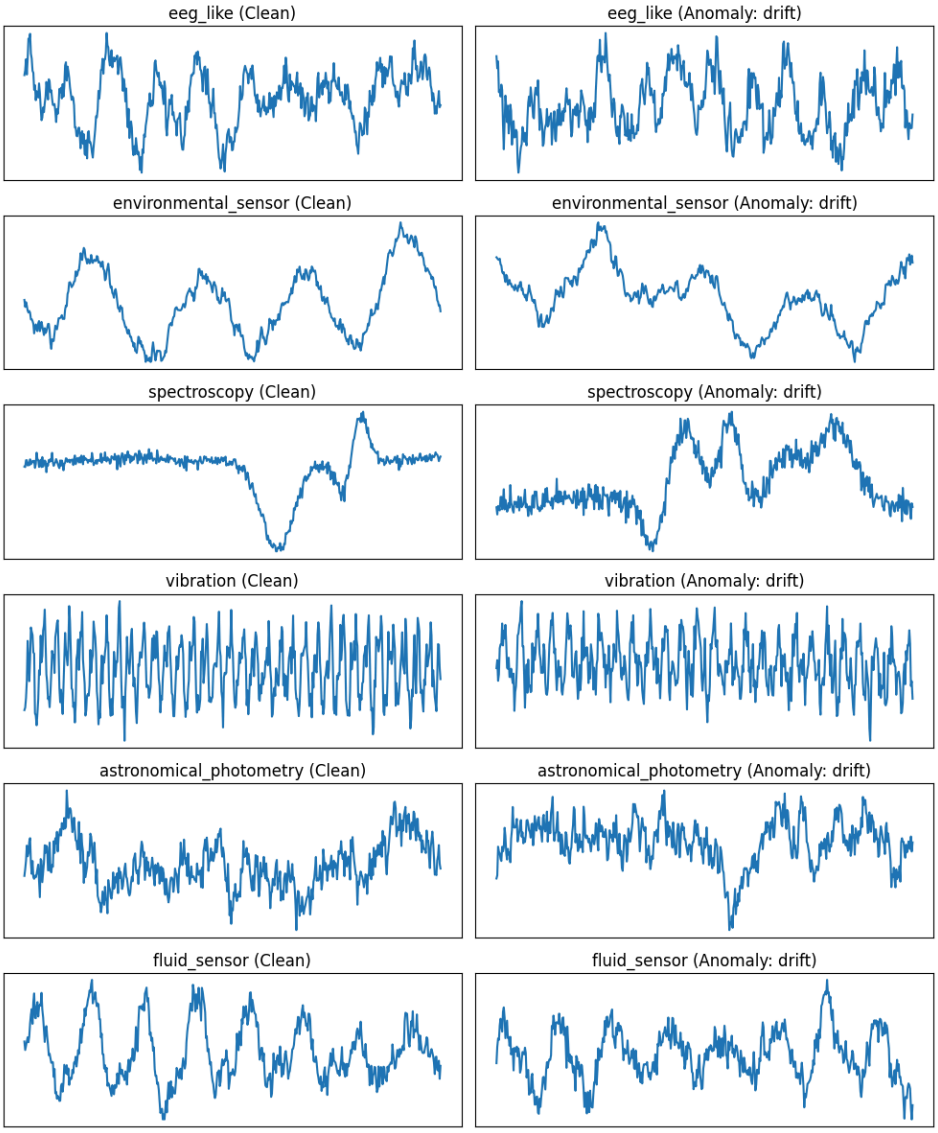


Figure 1. *Examples from the six synthetic scientific signal domains. The left column shows one clean trace per domain, and the right column shows an independently generated trace with an injected drift anomaly. The traces were selected deterministically with random seed 42. Because the clean and anomalous examples were sampled independently, each row illustrates domain morphology and the anomaly type rather than a pointwise before-and-after pair.*

3.2. Controlled Anomaly Mechanisms

Five anomaly mechanisms are injected into the clean signals:

- **Spike**: a short transient pulse added to the signal;
- **Drift**: a gradual baseline shift over part of the signal;
- **Dropout**: a flatline or attenuated region;
- **Oscillation disturbance**: a local high-frequency disturbance;
- **Distribution shift**: a local mean, variance, or scale change.

Each anomaly is evaluated across seven severity levels:

$$0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.35 \tag{1}$$

and four noise levels:

$$1.0,\ 2.0,\ 3.0,\ 5.0. \tag{2}$$

For each anomalous setting, 20 repeats are generated, giving $6 \times 5 \times 7 \times 4 \times 20 = 16,800$ anomalous traces. We independently generate 16,800 clean traces, balanced across the six domains and four noise levels. The final controlled dataset therefore contains 33,600 signals (50% clean and 50% anomalous), each with 512 time points. The extracted feature matrix contains the 56 SciPy-derived features listed in Table 2.

3.3. SciPy Feature Extraction

For each signal $x = [x_1, x_2, ..., x_T]$, we extract statistical, signal-processing, spectral, residual, local-window, and complexity features.

For the controlled 512-point signals, Savitzky-Golay smoothing used a 51-sample window, polynomial order 3, and `mode=interp`. Global Welch power spectral density estimates used `nperseg=128`; the local spectral features divided each signal into 128-sample windows with a 32-sample step and used `nperseg=64` within each window. The code left `noverlap` unspecified, so SciPy's default half-segment overlap gave 64 samples for the global estimates and 32 samples for the local estimates. For the 128-point windows in the UCR validation, the corresponding settings were a 31-sample Savitzky-Golay window with polynomial order 3 and `mode=interp`, and Welch `nperseg=64` with the default 32-sample overlap.

The core implementation calls `signal.savgol_filter(x, window_length=51, polyorder=3, mode="interp")` and computes the residual as `x - smooth`. Its energy, absolute mean, and maximum magnitude are then computed with NumPy reductions. For spectral features, the implementation calls `signal.welch(x, fs=fs, nperseg=128)` and integrates each selected frequency band with `simpson`. The same power spectral density supplies spectral entropy, dominant frequency, and high-frequency ratios.

Table 2. *The 56 extracted SciPy-derived features. Names match the notebook output columns.*

Category	**Feature names**
Statistical distribution	`mean`, `std`, `variance`, `min`, `max`, `range`, `median`, `iqr`, `mad`, `skewness`, `kurtosis`, `hist_entropy`, `normaltest_stat`, `normaltest_p`, `shapiro_stat`, `shapiro_p`
Amplitude and energy	`abs_mean`, `rms`, `energy`, `line_length`
Peak and transient	`peak_count`, `peak_density`, `mean_peak_prominence`, `max_peak_prominence`, `mean_peak_width`, `derivative_max`, `second_derivative_max`
Trend and residual	`savgol_residual_energy`, `savgol_residual_abs_mean`, `savgol_residual_max`, `rolling_mean_max_change`
Dropout and flatness	`rolling_var_min`, `rolling_var_max`, `rolling_var_ratio`, `flatness_fraction`
Spectral frequency	`delta_power`, `theta_power`, `alpha_power`, `beta_power`, `gamma_power`, `delta_ratio`, `theta_ratio`, `alpha_ratio`, `beta_ratio`, `gamma_ratio`, `spectral_entropy`, `dominant_frequency`, `high_frequency_ratio`, `local_high_freq_ratio_max`, `local_high_freq_ratio_std`, `local_spectral_entropy_min`, `local_spectral_entropy_std`
Complexity	`zero_crossing_rate`, `hjorth_activity`, `hjorth_mobility`, `hjorth_complexity`

3.4. Relative Feature-Group Signatures

To test interpretability, features are grouped into seven feature groups:

- statistical distribution;
- amplitude-energy;
- peak-transient;
- trend-residual;

- dropout-flatness;
- spectral-frequency;
- complexity.

For each domain, clean signals define a robust baseline. For each feature, we compute a robust absolute z-score relative to the clean baseline of the same domain, using the domain-specific median and 1.4826 times the median absolute deviation. Absolute z-scores are clipped at 8.0 to reduce domination by large-magnitude features. Within each feature group, the median clipped z-score is used as the group response. The group responses are then normalized so that they sum to one. This produces a relative feature-group signature for each sample. Of the 56 extracted features, 53 enter these groups; `normaltest_p`, `shapiro_p`, and `rolling_var_max` are retained in the full feature vectors but excluded from the relative signatures.

As implemented in the reported experiment, the domain baselines are estimated once from all 16,800 clean traces before the train-test split, rather than from the training partition alone. Thus test-set clean traces contribute to the robust centering and scaling used by the relative-signature representation; the full-feature and raw-signal representations do not use this baseline transformation.

The goal is to test whether anomaly mechanisms map to distinct relative feature-group patterns.

3.5. Models and Baselines

We evaluate three representations:

- full SciPy feature vectors;
- compressed relative feature-group signatures;
- raw time-domain signals.

For binary anomaly detection, we evaluate Logistic Regression, SVM-RBF, Random Forest, and Extra Trees models where applicable. For anomaly-type classification, we compare SciPy-feature models, relative-signature models, and raw-signal models. Both tasks use a row-level, stratified random split with 75% of rows for training and 25% for testing (random seed 42). The split is not grouped by repeat or domain. Only the separate leave-one-domain-out (LODO) evaluation holds out complete domains.

Table 3 records the non-default model settings used in the controlled and UCR experiments. Parameters not shown retain their scikit-learn defaults. Logistic Regression, SVM-RBF, and One-Class SVM inputs are standardized; the UCR feature and raw-window inputs are robustly standardized from their normal training windows.

Table 3. *Model hyperparameters used in the reported experiments.*

Model	Use	Hyperparameters
Logistic Regression	Controlled	`max_iter=3000`, `class_weight=balanced`, seed 42
SVM-RBF	Controlled binary	RBF kernel, $C = 1$, `gamma=scale`, `probability=True`, `class_weight=balanced`, seed 42
Random Forest	Controlled	300 trees for binary detection; 400 trees for anomaly-type classification; `class_weight=balanced`, seed 42, all CPU cores
Extra Trees	Controlled	300 trees for binary detection; 400 trees for anomaly-type classification and LODO; `class_weight=balanced`, seed 42, all CPU cores

Isolation Forest	UCR	200 trees, `contamination=0.02`, seed 42, all CPU cores; decision threshold is the 99th percentile of normal-training anomaly scores
One-Class SVM	UCR	RBF kernel, `gamma=scale`, $\nu = 0.02$; decision threshold is the 99th percentile of normal-training anomaly scores

The main evaluation metrics are balanced accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC for binary anomaly detection. For anomaly-type classification, we report accuracy, balanced accuracy, macro-F1, and weighted-F1.

3.6. Rule-Based Diagnosis

To test whether feature groups directly support mechanistic diagnosis, we define a simple rule-based diagnosis system:

- spike: peak-transient, statistical-distribution, and trend-residual groups;
- drift: trend-residual and statistical-distribution groups;
- dropout: dropout-flatness and trend-residual groups;
- oscillation disturbance: spectral-frequency and complexity groups;
- distribution shift: statistical-distribution and amplitude-energy groups.

For each anomalous signal, the rule system computes the mean response of the expected groups for each anomaly type and predicts the type with the largest score. If simple feature-group interpretability is sufficient, this rule system should classify anomaly type above chance.

4. Results

4.1. Binary Detection Was Trivial Under Controlled Injection

Perfect controlled binary scores show that the injected perturbations are measurable, but also that this synthetic boundary is too easy. Table 4 therefore motivates the harder mechanism task rather than a claim of deployment performance.

Table 4. *Binary anomaly detection results.*

Model	Balanced Acc.	Accuracy	F1	ROC-AUC	PR-AUC
SciPy Features + Random Forest	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Relative Signatures + Logistic Regression	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
SciPy Features + Extra Trees	0.9995	0.9995	0.9995	1.0000	1.0000
SciPy Features + SVM-RBF	0.9087	0.9087	0.9053	0.9721	0.9761
SciPy Features + Logistic Regression	0.8260	0.8260	0.8241	0.9167	0.9279
Raw Signal + Random Forest	0.7073	0.7073	0.6650	0.7926	0.8166
Raw Signal + Extra Trees	0.7060	0.7060	0.6632	0.7993	0.8198
Raw Signal + SVM-RBF	0.7015	0.7015	0.6475	0.7914	0.8082
Raw Signal + Logistic Regression	0.5039	0.5039	0.4973	0.5093	0.5295

Raw-signal tree baselines reached balanced accuracy near 0.71, showing that the SciPy representation makes the controlled boundary more separable.

4.2. Anomaly Diagnosis Was Substantially Harder

Each mechanism contributes 3,360 anomalous traces and 840 traces to the stratified test set (4,200 total). The best model reached macro-F1 0.4578, above the raw-signal baseline (0.2051) and most-frequent dummy (0.0667). The dummy's balanced accuracy is 0.20 but its macro-F1 is lower because it predicts only one class. Table 5 reports all results.

Table 5. *Anomaly-type classification results across five anomaly mechanisms.*

Model	Balanced Acc.	Accuracy	Macro-F1	Support
SciPy Features + Random Forest	**0.4660**	**0.4660**	**0.4578**	840/class
SciPy Features + Extra Trees	0.3852	0.3852	0.3896	840/class
Relative Signatures + Logistic Regression	0.2160	0.2160	0.2070	840/class
Relative Signatures + Extra Trees	0.2057	0.2057	0.2051	840/class
Raw Signal + Extra Trees	0.2050	0.2050	0.2051	840/class
Most-frequent dummy	0.2000	0.2000	0.0667	840/class

4.3. Confusion Structure of Anomaly Mechanisms

The macro-F1 score shows that anomaly-mechanism classification is substantially harder than binary detection, but it does not reveal which mechanisms are confused. We therefore inspect the confusion matrix of the best anomaly-type classifier, SciPy Features + Random Forest. Table 6 shows the row-normalized confusion matrix, where each row represents the true anomaly type and each column represents the predicted anomaly type.

Table 6. *Row-normalized confusion matrix for anomaly-type classification using SciPy Features + Random Forest. Rows indicate true anomaly type and columns indicate predicted anomaly type.*

True / Predicted	Distribution Shift	Drift	Dropout	Oscillation Disturbance	Spike
Distribution Shift	0.34	0.17	0.07	0.28	0.13
Drift	0.07	0.72	0.03	0.12	0.06
Dropout	0.06	0.09	0.60	0.10	0.15
Oscillation Disturbance	0.21	0.04	0.11	0.45	0.18
Spike	0.24	0.10	0.18	0.26	0.21

The confusion matrix shows that the classifier learned some anomaly mechanisms better than others. Drift was the most separable mechanism, with 72% of drift samples correctly classified. Dropout was also partially separable, with 60% correctly classified. Oscillation disturbance was detected at a lower but still meaningful rate, with 45% correctly classified. In contrast, distribution shift and spike were much less separable. Distribution shift was correctly classified only 34% of the time and was often confused with oscillation disturbance. Spike was the weakest class, with only 21% correctly classified, and was frequently confused with distribution shift and oscillation disturbance.

This confusion structure supports the paper's central argument: SciPy features contain partial mechanism-level information, but anomaly diagnosis is not solved by global feature summaries. Drift and dropout produce more consistent feature patterns, likely because they alter baseline or flatness structure. Spike and oscillation disturbance are more ambiguous in the current representation, suggesting that short local transients and local spectral changes are not sufficiently isolated by whole-signal features. This reinforces the need for local anomaly localization before mechanism diagnosis.

This pattern has a mechanistic explanation. Drift produces a sustained baseline displacement, which changes rolling mean structure and creates a cumulative low-frequency effect across a large portion of the signal. Because the perturbation persists over many time steps, global and rolling-window features can capture it reliably. Dropout similarly creates an extended low-variance or attenuated region, which is partly captured by rolling-variance and flatness features. Spike is different: it is a short local transient. When features are computed over the full $T = 512$ point segment, the contribution of a narrow spike is diluted by the segment length. A local spike may strongly affect a few derivative or peak features, but many global statistics average it with hundreds of normal samples. This explains why spike was frequently confused with distribution shift and oscillation disturbance. It also suggests that reliable spike diagnosis requires anomaly localization or local-window feature extraction before whole-signal classification.

4.4. Relative Feature-Group Signature Map

Table 7 shows the average relative feature-group signature for each anomaly type. The strongest group for every anomaly type was amplitude-energy.

Table 7. *Relative feature-group signature map by anomaly type.*

Anomaly Type	Stat.	Amp./Energy	Peak	Trend	Dropout	Spectral	Complexity
Distribution shift	0.1046	**0.5383**	0.0745	0.0855	0.0572	0.0598	0.0801
Drift	0.1069	**0.5379**	0.0733	0.0835	0.0583	0.0599	0.0803
Dropout	0.1088	**0.5279**	0.0756	0.0833	0.0663	0.0612	0.0770
Oscillation disturbance	0.1044	**0.5403**	0.0735	0.0844	0.0583	0.0599	0.0792
Spike	0.1046	**0.5407**	0.0738	0.0846	0.0566	0.0602	0.0796

The signature map does not support the simple hypothesis that each anomaly type maps cleanly to a different dominant feature group. Even after robust normalization, clipping, median aggregation, and relative normalization, amplitude-energy remained the dominant group for every anomaly type. This indicates that the controlled anomaly injections often changed global signal energy even when the intended anomaly mechanism was local, spectral, or structural.

As a post-hoc no-energy control, we removed the amplitude-energy column from the mean signatures in Table 7 and renormalized the six remaining columns to sum to one. Statistical-distribution then became the largest mean group for all five mechanisms, with normalized shares between 0.227 and 0.231. Thus removing energy changes which group dominates but does not recover a distinct dominant group for each mechanism. This control is descriptive; we did not retrain the classifiers or recompute per-trace rule accuracy without the energy group.

4.5. Rule-Based Diagnosis

The rule-based diagnosis system achieved overall accuracy = 0.1993, essentially equal to the 0.2000 accuracy of a most-frequent-class baseline on the balanced five-class dataset. Table 8 shows the accuracy by anomaly type and makes clear that the aggregate result comes almost entirely from predicting distribution shift.

Table 8. *Rule-based diagnosis accuracy by anomaly type.*

Anomaly Type	Rule Accuracy

Distribution shift	0.9881
Drift	0.0006
Dropout	0.0077
Oscillation disturbance	0.0000
Spike	0.0000

The rule-based diagnosis result reveals the central failure mode of naive interpretability. Distribution shift achieved 0.9881 rule accuracy, while spike, drift, dropout, and oscillation disturbance were nearly always misdiagnosed. This pattern is not random. The rule system assigns distribution shift using statistical-distribution and amplitude-energy responses. Since amplitude-energy dominated the relative signature map for every anomaly type, the rule system was biased toward the distribution-shift rule. As a result, distribution shift was diagnosed correctly, not because the rule system learned a rich mechanism-specific signature, but because the confounding amplitude-energy response made other anomaly types resemble distribution-shift behavior.

This explains why the overall rule-based diagnosis accuracy was approximately chance. The rule system did not fail because the features were meaningless. It failed because a globally responsive feature group collapsed the signature space. This is a precise example of why named features alone are not sufficient for scientific interpretability.

4.6. Leave-One-Domain-Out Generalization

To test domain generalization, we trained an anomaly-type classifier on five domains and tested on the held-out sixth domain. Table 9 shows the results.

Table 9. *Leave-one-domain-out anomaly-type classification.*

Held-Out Domain	Balanced Acc.	Macro-F1
EEG-like	0.4114	0.4194
Fluid sensor	0.3939	0.3941
Astronomical photometry	0.3800	0.3728
Environmental sensor	0.3418	0.3503
Vibration	0.3307	0.3291
Spectroscopy	0.2611	0.2087

The leave-one-domain-out results show partial but limited domain generalization. Some domains, such as EEG-like and fluid-sensor signals, generalized better than others. Spectroscopy and vibration were more difficult. This suggests that anomaly mechanisms are not fully domain-independent. The same injected anomaly may interact differently with different baseline signal structures.

4.7. Statistical Tests on Feature-Group Responses

Kruskal-Wallis tests were used to test whether feature-group responses differed by anomaly type. Table 10 shows the results.

Table 10. *Kruskal-Wallis tests for feature-group response differences by anomaly type.*

Feature Group	H Statistic	p-value
Statistical distribution	56.2222	1.80×10^{-11}
Dropout flatness	36.5159	2.27×10^{-7}

Amplitude-energy	26.7147	2.27×10^{-5}
Complexity	9.9531	4.12×10^{-2}
Spectral frequency	6.3235	0.1763
Trend residual	6.2548	0.1809
Peak transient	5.4034	0.2484

Each Kruskal-Wallis test used all 16,800 anomalous traces, with five equally sized mechanism groups of $n = 3,360$. Seven feature-group tests were performed. With an uncorrected $\alpha = 0.05$, statistical-distribution, dropout-flatness, amplitude-energy, and complexity differed across mechanisms. Under a Bonferroni correction, the threshold is $0.05/7 = 0.00714$; only statistical-distribution, dropout-flatness, and amplitude-energy remain significant, while complexity ($p = 0.0412$) does not. Spectral-frequency, trend-residual, and peak-transient are not significant under either threshold. Thus anomaly-type differences exist, but they are not captured by a simple one-group-per-anomaly interpretation.

An important irony is that the peak-transient and trend-residual groups, which are the most intuitively matched to spike and drift anomalies, did not reach Kruskal-Wallis significance in the 16,800-trace controlled analysis. This reinforces the central argument that intuitive feature names do not automatically translate into reliable diagnostic separation.

5. Real-Data Validation on the UCR Time Series Anomaly Archive

5.1. Dataset and Setup

To evaluate whether the SciPy anomaly detection workflow transfers beyond controlled synthetic injection, we performed an external validation experiment on the 2021 UCR Time Series Anomaly Archive [1], using `UCR_TimeSeriesAnomalyDatasets2021.zip` from the archive URL recorded in the notebook. The notebook does not record a download date. Unlike the controlled benchmark, where anomaly type, severity, and noise level are generated by design, the UCR archive contains real time-series anomaly problems with labeled anomaly intervals encoded in the file metadata.

We evaluated the first 60 archive files in lexicographic filename order. Four files were skipped because the sliding-window labeling procedure produced no anomalous test windows, leaving 56 evaluated files. Table 11 identifies both sets. Each time series was converted into overlapping windows of length 128 with step size 16. Windows ending before the training boundary were used as normal training windows. Test windows were labeled anomalous if they overlapped the labeled anomaly interval by at least 10% of the window length. Point-adjusted F1 treats an anomaly interval as detected if at least one overlapping window is flagged, which is common in time-series anomaly detection but can be more optimistic than raw window-level F1.

Table 11. *UCR file selection. Numeric IDs are the filename prefixes in the 2021 archive.*

Status	Archive file IDs or filenames
Evaluated (56)	001–004, 006–026, 028–029, 031–050, and 052–060
Skipped (4)	`005_UCR_Anomaly_DISTORTEDCIMIS44AirTemperature1_4000_5391_5392.txt`; `027_UCR_Anomaly_DISTORTEDInternalBleeding16_1200_4187_4199.txt`; `030_UCR_Anomaly_DISTORTEDInternalBleeding19_3000_4187_4197.txt`; `051_UCR_Anomaly_DISTORTEDTkeepSecondMARS_3500_9330_9340.txt`

For each window, we extracted the same SciPy-inspired statistical and signal-processing features used in the controlled experiment, including distributional statistics, peak features, residual features, rolling-window features, and spectral features. We evaluated four unsupervised methods: SciPy Features + Robust Distance, SciPy Features + Isolation Forest, SciPy Features + One-Class SVM, and Raw Windows + Isolation Forest. We do not claim state-of-the-art performance on the UCR archive. The purpose of the UCR experiment is external validation of the SciPy feature workflow under naturally labeled anomaly intervals, not benchmark optimization.

5.1.1. *Methodological asymmetry.*

The controlled benchmark and UCR validation evaluate different learning settings. In the controlled benchmark, anomaly labels and anomaly-mechanism labels are known by construction, so supervised classifiers can be trained for both binary detection and anomaly-type classification. In contrast, the UCR archive provides normal training regions and labeled anomaly intervals for evaluation, but it does not provide labeled examples of each anomaly mechanism for supervised mechanism classification. Therefore, the UCR validation is evaluated using unsupervised and one-class methods trained on normal windows. This asymmetry is intentional: the controlled benchmark tests mechanism-level hypotheses under known labels, while the UCR experiment tests whether the SciPy feature workflow remains useful when only normal training data are available. The two experiments should therefore not be interpreted as identical evaluations, but as complementary tests of mechanism understanding and real-data transfer.

5.2. Results

Table 12 summarizes the real-data validation results. The strongest method by mean point-adjusted F1 was SciPy Features + Isolation Forest, which achieved mean point-adjusted F1 = 0.2884, raw F1 = 0.1248, ROC-AUC = 0.7758, PR-AUC = 0.1801, and event detection rate = 0.4107 across 56 evaluated files. SciPy Features + One-Class SVM achieved the highest event detection rate at 0.6607, but had a lower mean point-adjusted F1 of 0.2646.

Table 12. *Real-data validation on the UCR Time Series Anomaly Archive.*

Method	Raw Bal. Acc.	Raw F1	ROC-AUC	PR-AUC	Point-Adj. F1	Event Detection
SciPy Features + Isolation Forest	0.5588	0.1248	**0.7758**	0.1801	**0.2884**	0.4107
SciPy Features + One-Class SVM	**0.6374**	**0.1375**	0.6561	**0.2143**	0.2646	**0.6607**
Raw Windows + Isolation Forest	0.5484	0.1184	0.5450	0.1582	0.2617	0.3571
SciPy Features + Robust Distance	0.5340	0.0877	0.7225	0.1331	0.2328	0.2857

Each selected UCR file contains one labeled anomaly interval, and the notebook saved only whether that interval was hit by at least one positive window. It did not save the contigu-

ous predicted event intervals needed to count false event-level alarms. Consequently, the reported event quantity is a detection rate (equivalently recall over the 56 labeled events), while event-level precision cannot be reconstructed from the saved run. Raw window-level precision, included in the notebook's per-file results, measures false positive windows but is not a substitute for event-level precision.

The UCR results show a sharp contrast with the controlled synthetic benchmark. In the synthetic experiment, binary anomaly detection was nearly trivial, with SciPy-feature models reaching perfect performance. In the real-data validation experiment, performance was much lower. This confirms that perfect performance under controlled injection should not be interpreted as deployment-level anomaly detection ability. Real anomaly intervals are more heterogeneous, less aligned with fixed window boundaries, and often subtler than injected perturbations.

The real-data results also show that different metrics emphasize different operational goals. SciPy Features + Isolation Forest achieved the best point-adjusted F1 and ROC-AUC, suggesting better ranking and localization performance. SciPy Features + One-Class SVM achieved the highest event detection rate, suggesting that it was more likely to flag at least one window inside the anomaly interval. Depending on the scientific workflow, detecting an event may be more important than window-level precision if the goal is to alert a researcher that a suspicious episode occurred, but the missing event-level precision prevents a complete event-based comparison.

5.3. Best and Worst Cases

The SciPy Features + Isolation Forest method performed strongly on some UCR files but failed completely on others. Table 13 gives the exact per-file scores underlying those examples.

Table 13. *Selected best and worst per-file results for SciPy Features + Isolation Forest. All worst-case rows have point-adjusted F1 = 0, raw F1 = 0, and no detected event.*

ID	Series	Point-Adj. F1	Raw F1	Event Detected
004	BIDMC1	0.9500	0.1739	Yes
010	CIMIS44AirTemperature6	0.9474	0.1818	Yes
044	PowerDemand1	0.9153	0.4000	Yes
042	Lab2Cmac011215EPG6	0.9130	0.7805	Yes
008	CIMIS44AirTemperature4	0.9091	0.4000	Yes
037	Lab2Cmac011215EPG1	0.9000	0.3077	Yes
028	InternalBleeding17	0.8485	0.5385	Yes
007	CIMIS44AirTemperature3	0.8421	0.4286	Yes
045	PowerDemand2	0.8169	0.3529	Yes
054	WalkingAceleration5	0.7692	0.4762	Yes
046	PowerDemand3	0.0000	0.0000	No
047	PowerDemand4	0.0000	0.0000	No
049	TkeepFirstMARS	0.0000	0.0000	No
050	TkeepForthMARS	0.0000	0.0000	No
055	apneaecg2	0.0000	0.0000	No
056	apneaecg3	0.0000	0.0000	No
057	apneaecg4	0.0000	0.0000	No

058	apneaecg	0.0000	0.0000	No
059	gait1	0.0000	0.0000	No
060	gait2	0.0000	0.0000	No

This variation is important. It shows that SciPy feature-based anomaly detection is not universally reliable across real time-series domains. Some anomaly intervals produce feature deviations that are detectable by the current window-level feature representation, while others do not. This supports the paper's broader conclusion: SciPy features are useful and interpretable, but real scientific anomaly detection requires domain calibration, careful thresholding, and possibly local or task-specific feature design.

6. Discussion

Controlled binary detection was perfect, but mechanism diagnosis reached only macro-F1 0.4578 and the best UCR method reached 0.2884 mean point-adjusted F1. Controlled injection therefore tests whether mechanisms are measurable, not deployment readiness. Real anomalies add heterogeneous shapes, baselines, and thresholding conditions, so scientific use requires domain-specific validation.

6.1. Naive Interpretability Failed

The rule-based system tested whether intuitive groups directly diagnose mechanisms, but achieved chance accuracy because amplitude-energy dominated every average signature. Named features can be understandable in isolation without providing reliable diagnosis.

6.2. Why Global Energy Features Confound Diagnosis

For a clean signal x, perturbation δ, and squared-energy function $E(x) = \sum_t x_t^2$, the energy change is

$$E(x+\delta) - E(x) = 2\sum_t x_t \delta_t + E(\delta). \tag{3}$$

Spikes, oscillations, and shifts often increase energy, while attenuation can decrease it. Absolute deviations in RMS, variance, and energy therefore respond to many mechanisms: they detect magnitude change without identifying its cause.

6.3. SciPy Features Still Preserve Mechanism Information

Full SciPy features nevertheless outperformed the raw-signal baseline (macro-F1 0.4578 versus 0.2051), suggesting that mechanism information is multivariate and overlapping rather than confined to one group.

6.4. Domain Generalization Is Partial

LODO results confirm domain dependence: spectroscopy already contains peaks and vibration already contains oscillations, making corresponding anomalies ambiguous. SciPy features are a useful representation, not universal anomaly signatures.

7. Reproducibility

All generation, feature extraction, model training, evaluation, and UCR windowing code is available in a public executable Colab notebook at https://colab.research.google.com/drive/1dEcrG2ynX3k_dAHSWQmvjt42jhMRYc8S?usp=sharing; the submitted local copy is `anamolydetection.ipynb`. The experiments used Python with NumPy [7], SciPy [2], scikit-learn [8], pandas [9], and Matplotlib [10]. A fixed random seed of 42 was used for controlled signal

generation, train-test splitting, and model initialization where supported. Each controlled signal contained 512 time points. The controlled experiment used six generated scientific signal domains, five injected anomaly mechanisms, seven severity levels, four noise levels, and 20 anomaly repeats per setting. It contained 16,800 anomalous and 16,800 independently generated clean traces. The UCR validation used sliding windows of length 128 with step size 16, and test windows were labeled anomalous if at least 10% of the window overlapped the labeled anomaly interval.

8. Limitations

The synthetic generators and single-mechanism perturbations do not reproduce all instrument faults, combined anomalies, missingness, or nuisance shifts. The controlled binary task is too easy, and the all-clean baseline used for relative signatures leaks test-distribution information. The controlled supervised and UCR one-class scores are complementary rather than directly comparable. The UCR archive does not replace instrument-specific validation, and the saved predictions cannot support event-level precision.

9. Future Work

Future work should fit every transformation on training data, use harder and overlapping perturbations, and validate on fault-labeled instruments. A two-stage system could localize a suspicious interval before computing mechanism-specific features, preventing short events from being diluted across 512 points. User studies should test whether explanations help identify mechanisms, and limited-data comparisons should include deep models.

10. Conclusion

SciPy features perfectly separated controlled clean and anomalous signals, but mechanism diagnosis remained difficult and rule-based explanations performed at chance. Mean point-adjusted F1 of 0.2884 on UCR further limits the claim. SciPy features are a transparent baseline, not a complete solution; scientific workflows must distinguish detection, diagnosis, and real-data transfer.

References

[1] R. Wu and E. J. Keogh, "Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 3, pp. 2421–2429, 2023, doi: 10.1109/TKDE.2021.3112126.

[2] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[4] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the Support of a High-Dimensional Distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001, doi: 10.1162/089976601750264965.

[5] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection." 2016. doi: 10.48550/arXiv.1607.00148.

[6] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data," *Proceedings of the VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, 2022, doi: 10.14778/3514061.3514067.

[7] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011, doi: 10.48550/arXiv.1201.0490.

[9] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[10] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.